

C basics

Lecture 1

Subject: Logical Thinking & Problem Solving

Division of Technical Training
Engineering Foundations - UIE

Mr. Vivek Singh

Technical Trainer

Division of Technical Training

C is a powerful, structured programming language that was developed by Dennis Ritchie in 1972 at Bell Labs. It is used to write software that interacts closely with hardware, like operating systems (e.g., UNIX), device drivers, and compilers.

It is often called the "mother of all programming languages" because many modern languages like C++, Java, and Python are either built on C or influenced by its structure and logic.

Why Use C?

- It's fast and efficient.
- You get full control over memory — useful in system-level programming.
- It's the foundation of many other modern programming languages.
- It's still used in real-world systems like microcontrollers, game engines, and operating systems

Features of C language

- **Middle-Level Language:** Combines the simplicity of high-level languages with the power of low-level programming.
- **Simple and Efficient:** Small language with a limited set of keywords, making it easy to learn and fast to execute.
- **Structured Programming:** Supports functions, loops, conditional statements, and modular program design.
- **Portable:** Programs written in C can run on different computers and operating systems with little or no modification.
- **Rich Standard Library:** Provides many built-in functions for input/output, mathematics, string handling, file operations, and more.
- **Extensible:** Users can create and reuse their own functions and libraries.
- **Fast Execution:** Produces efficient machine code, making it suitable for performance-critical applications.

Applications of C language

- **Operating Systems:** Used to develop operating systems such as UNIX and major parts of Linux.
- **Embedded Systems:** Programming microcontrollers, IoT devices, and consumer electronics.
- **System Software:** Used for developing compilers, interpreters, device drivers, and utilities.
- **Application Software:** Used to build desktop applications, business software, and scientific programs.
- **Database Systems:** Many database management systems are developed using C for high performance.
- **Networking Software:** Used to develop network protocols and communication software.
- **Game Development:** Forms the foundation of many game engines and graphics applications.
- **Scientific & Engineering Applications:** Widely used for simulations, mathematical computations, and high-performance computing.

'stdio.h' which is the most important header file as it include the functions which are necessary for input-output operations.

Structure of writing a C program

Curly braces : Define the starting and ending boundaries of your program's blocks.

```
/* # is a preprocessor directive it directs the  
preprocessor to pre process the code inside the  
header file mentioned in angle brackets before  
the compilation begins so that you could use all  
the functions which are defined inside the file  
*/
```

```
#include<stdio.h>
```

```
/* Main is entry function to your C code which will  
be called by the compiler and rest any function which  
you wanna use will be either directly or indirectly  
called through this function */
```

```
int main(){ // int is return type and main is the name of the function  
    printf("Hello world"); // printf is a built in function inside stdio.h  
    /* we always return 0 from main function as  
    it indicates function executed successfully*/  
    return 0;  
}
```

basic building blocks of C:

- Preprocessor directives
- Header files
- Character set
- Keywords
- Identifiers
- Variables
- Constants
- Operators
- Data types and their storage

It is used to print the statement on the screen.

C Character Set

: The fundamental characters (alphabets, digits, symbols, spaces) that C allows to build valid expressions and statements. All characters map directly to standard **ASCII values**.

Alphabets 52 total characters

A, B, C ,Z

a, b, c ,z

Digits 10 total characters

0 1, 2, 3, 4, 5, 6, 7, 8, 9,

C Character Set

Special Characters

Character	Meaning	Character	Meaning
+	plus sign	-	minus sign(hyphen)
*	asterisk	%	percent sign
\	Backward slash	/	forward slash
<	less than sign	=	equal to sign
>	greater than sign	_	underscore
(	left parenthesis	)	right parenthesis
{	left braces	}	right braces
[	left bracket	]	right bracket
,	comma	.	period
'	single quotes	"	double quotes
:	colon	;	Semicolon
?	Question mark	!	Exclamation sign
&	ampersand		vertical bar
@	at the rate	^	caret sign
\$	dollar sign	#	hash sign
~	tilde sign	`	back quotation mark

C Character Set

An **escape sequence** is a pair of characters starting with a backslash (\) that tells the computer to perform a special action—like jumping to a new line or adding a tab space

Escape sequences

Escape Sequence	Meaning	ASCII Value	Purpose
\b	backspace	008	Moves the cursor to the previous position of the current line
\a	bell(alert)	007	Produces a beep sound for alert
\r	carriage return	013	Moves the cursor to beginning of the current line.
\n	newline	010	Moves the cursor to the beginning of the next line
\f	form feed	012	Moves the cursor to the initial position of the next logical page.
\0	null	000	Null
\v	vertical tab	011	Moves the cursor to next vertical tab position
\t	Horizontal tab	009	Moves the cursor to the next horizontal tab position.
\\	backslash	092	Presents a character with backslash (\)

C Character Set

Delimiters

Delimiters are special characters or sequences of characters used to mark the beginning, end, or separation of distinct units of data, code statements, or text blocks.

:	colon	used for label
;	semicolon	end of statement
()	parentheses	used in expression
[]	square brackets	used for array
{ }	curly braces	used for block of statements
#	hash	preprocessor directive
,	comma	variable delimiter

The smallest individual units in a C program that the compiler can recognize and interpret without splitting them further

tokens

Reserved Words / Keywords

Predefined, reserved words with special meaning to the compiler

auto	break	case	char
const	continue	default	do
double	else	enum	extern
float	for	goto	if
int	long	register	return
short	signed	sizeof	static
struct	switch	typedef	union
unsigned	void	volatile	while

Step-by-Step Explanation (The Six Tokens)

15. 1. **Keywords:** Reserved words that have special pre-defined meanings in C (e.g., int, float, return,

switch). You cannot change their meaning.

16. 2. **Identifiers:** Names given to variables, arrays, or functions (e.g., main, age, total_sum). Rules for naming identifiers:

- Must start with an alphabet letter or an underscore (_).
- Cannot contain spaces or special characters (except _).
- Cannot be a keyword.

17. 3. **Constants:** Fixed values that cannot be modified during program execution (e.g., 3.1415, 100).

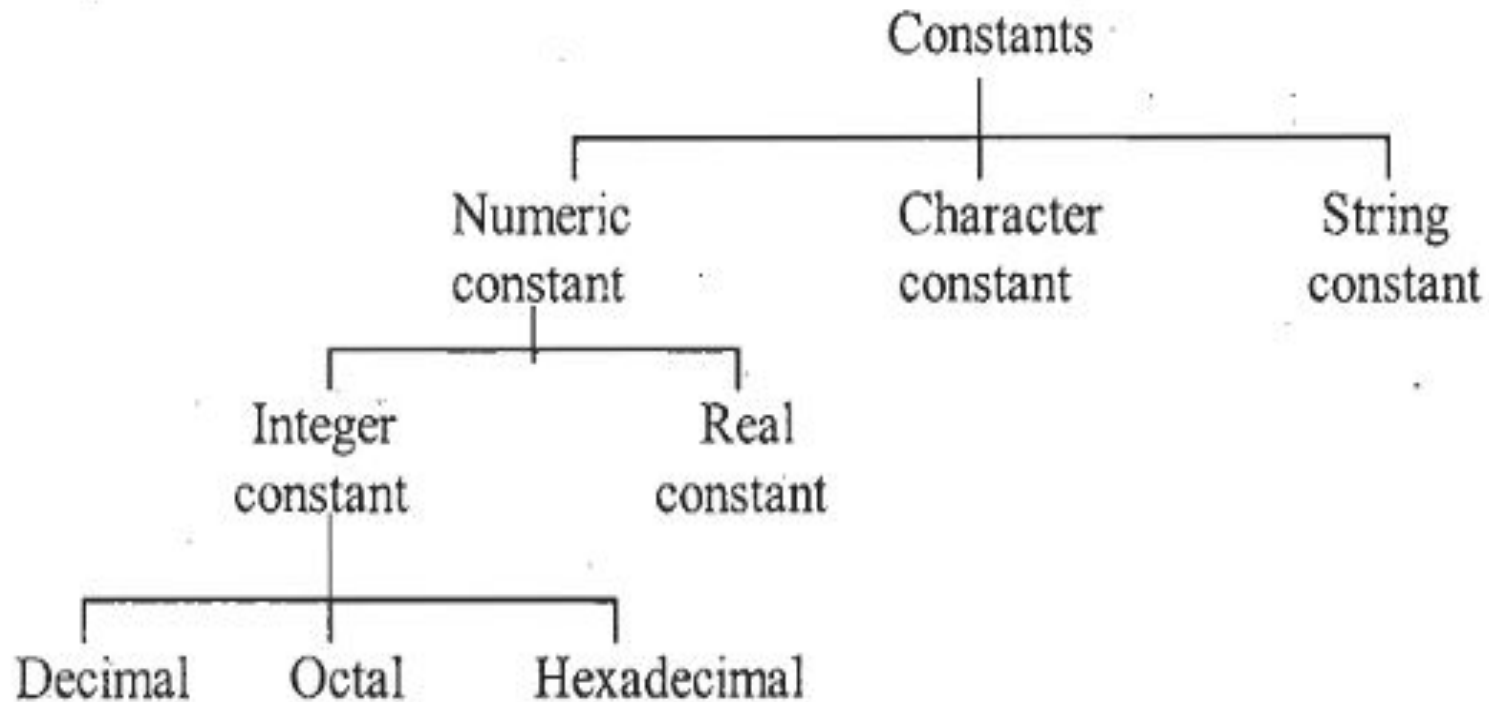
18. 4. **Strings:** Text sequences wrapped inside double quotes (e.g., "Hello Topper").

19. 5. **Operators:** Symbols that perform mathematical or logical operations (e.g., +, -, *, /, &&).

20. 6. **Special Symbols:** Syntactic punctuation characters (e.g., {, }, (,), [,], :, ;, ,).

tokens

Constants A **constant** is a fixed value in a program that **cannot be changed** during the execution of the code.



Integer Constant: A fixed **whole number** without any decimal points or fractional parts (e.g., 10, -5, 0).

Real Constant: A fixed number that **contains a decimal point** or fractional part (e.g., 3.14, -0.5, 99.9)

A single letter, digit, or symbol enclosed inside single quotes (' ') : character constant

Any group of characters enclosed within " " is known as string constant

Variables

Variable is a name that can be-used to store values. Variables can take different values but one at a time. These values can be changed during execution of the program. A data type is associated with each variable. The data type of the variable decides what values it can take. The rules for naming variables are same as that for naming identifiers.

Rules for Identifiers

- An identifier can contain **letters (A–Z, a–z), digits (0–9), and the underscore (_)** only.
- The **first character** must be a **letter or an underscore (_)**. It **cannot start with a digit**. But always try to avoid **(_) at beginning**.
- An identifier **cannot be a C keyword** (e.g., `int`, `while`, `return`).
- C is **case-sensitive**, so `count`, `Count`, and `COUNT` are treated as **different identifiers**.
- Identifiers can be **of any length**, but many compilers recognize only a limited number of significant characters (commonly **31 or more**).
- Use **meaningful and descriptive names** to improve code readability

Why avoid leading () in variable names

- Reserved for being used by the standard library and compiler
- For example all variable names beginning with () are reserved to be used by compiler
- Also all variable names beginning with () and capital letters are also reserved

Operators

Connectors used with operands(could be constants or variables) to enable you perform various operations

- Arithmetic: +, -, *, /, %
- Relational: ==, !=, <, >, <=, >=
- Logical: &&, ||, !
- Assignment: =, +=, -=, *=, /=, %=, &=, |=, ^=, <<=, >>=
- Increment/Decrement: ++, --
- Bitwise: &, |, ^, <<, >>
- Conditional: ?:
- sizeof operator
- TypeCast operator

Arithmetic operators

provides all basic
arithmetic operations

Operator	Symbol	Example (in C) and Resulting Value
Addition	+	$10 + 5 \rightarrow 15$
Subtraction	-	$10 - 5 \rightarrow 5$
Multiplication	*	$10 * 5 \rightarrow 50$
Division	/	$10 / 5 \rightarrow 2$
Modulus	%	$10 \% 3 \rightarrow 1$

Note: In C, '/' gives the quotient for integer operands. For example, $10 / 3$ gives 3, not 3.33.

A **compound assignment operator** (also called a shorthand assignment operator) is a single operator that combines an arithmetic or bitwise operation with an assignment operation in one step.

Instead of writing `x = x + 5`, you can write `x += 5`.

Compound assignment operators

Operator (Compound Arithmetic)	Symbol	Meaning	Simplified Version (Equivalent to)	Example (in C)	Resulting Value
Addition Assignment	<code>+=</code>	Add right operand to the left operand and assign the result to left operand	<code>a += b</code> is equivalent to <code>a = a + b</code>	<code>int a = 10;</code> <code>a += 5;</code>	<code>a = 15</code>
Subtraction Assignment	<code>-=</code>	Subtract right operand from the left operand and assign the result to left operand	<code>a -= b</code> is equivalent to <code>a = a - b</code>	<code>int a = 10;</code> <code>a -= 5;</code>	<code>a = 5</code>
Multiplication Assignment	<code>*=</code>	Multiply left operand with right operand and assign the result to left operand	<code>a *= b</code> is equivalent to <code>a = a * b</code>	<code>int a = 10;</code> <code>a *= 5;</code>	<code>a = 50</code>
Division Assignment	<code>/=</code>	Divide left operand by right operand and assign the result to left operand	<code>a /= b</code> is equivalent to <code>a = a / b</code>	<code>int a = 10;</code> <code>a /= 2;</code>	<code>a = 5</code>
Modulus Assignment	<code>%=</code>	Divide left operand by right operand and assign the remainder to left operand	<code>a %= b</code> is equivalent to <code>a = a % b</code>	<code>int a = 10;</code> <code>a %= 3;</code>	<code>a = 1</code>

- Other than these arithmetic compound assignment operators we also have bitwise compound assignment operators which we will discuss once we would be done with bitwise operators
- And as you all know we have simple assignment operator '`=`' as well for ex `a = 5`

Assignment operators are used to assign result of an expression to a variable.

• '`=`' is the assignment operator in C

Relational operators

Relational operator compares between two operands and returns in true and false.

Operator	Symbol	Example (in C) and Resulting Value
Equal to	==	5 == 5 → 1 (true)
Not equal to	!=	5 != 3 → 1 (true)
Greater than	>	7 > 3 → 1 (true)
Less than	<	7 < 3 → 0 (false)
Greater than or equal to	>=	7 >= 7 → 1 (true)
Less than or equal to	<=	4 <= 2 → 0 (false)

Note: In C, relational operators return 1 (true) if the condition is true, otherwise 0 (false).

Logical operators

Logical operator is used to compare or evaluate logical and relational expressions.

Operator	Symbol	Example (in C) and Resulting Value
Logical AND	&&	(5 > 3) && (2 < 4) → 1 (true) (5 > 3) && (2 > 4) → 0 (false)
Logical OR		(5 > 3) (2 > 4) → 1 (true) (5 < 3) (2 > 4) → 1 (true)
Logical NOT	!	!(5 > 3) → 0 (false) !(2 > 4) → 1 (true)
Logical AND (with values)	&&	(1 && 0) → 0 (false) (1 && 1) → 1 (true)
Logical OR (with values)		(1 0) → 1 (true) (0 0) → 0 (false)
Logical NOT (with value)	!	!1 → 0 (false) !0 → 1 (true)

Note: In C, logical operators return 1 (true) if the condition is true, otherwise 0 (false).

Increment/decrement operators

The increment operator ++ adds 1 to the operand.

Operator	Symbol	Example (in C) and Resulting Value
Increment (Prefix)	++variable	int a = 5; b = ++a; → a = 6, b = 6 (Value is incremented first, then used)
Increment (Postfix)	variable++	int a = 5; b = a++; → a = 6, b = 5 (Value is used first, then incremented)
Decrement (Prefix)	--variable	int a = 5; b = --a; → a = 4, b = 4 (Value is decremented first, then used)
Decrement (Postfix)	variable--	int a = 5; b = a--; → a = 4, b = 5 (Value is used first, then decremented)

The decrement operator -- subtracts 1 from the operand.

Note: In C, ++ increases the value by 1 and -- decreases the value by 1.

Bitwise operators are special operators used to perform calculations directly on the individual **bits** (0s and 1s) of integer data, rather than on the whole number as a single value.

When you use a bitwise operator, the computer converts the numbers into their binary form, performs the operation bit-by-bit (column by column), and converts the result back into a standard number.

Bitwise operators

Operator (Bitwise)	Symbol	Meaning	Example (in C)	Binary Example (Consider a = 12, b = 10)	Resulting Value
Bitwise AND	&	Performs AND operation on each pair of bits.	int a = 12, b = 10; int c = a & b;	a = 12 → 0000 1100 b = 10 → 0000 1010 ----- a & b = 0000 1000	c = 8
Bitwise OR		Performs OR operation on each pair of bits.	int a = 12, b = 10; int c = a b;	a = 12 → 0000 1100 b = 10 → 0000 1010 ----- a b = 0000 1110	c = 14
Bitwise XOR	^	Performs XOR operation on each pair of bits.	int a = 12, b = 10; int c = a ^ b;	a = 12 → 0000 1100 b = 10 → 0000 1010 ----- a ^ b = 0000 0110	c = 6
Bitwise NOT	~	Inverts (complements) each bit (0 becomes 1 and 1 becomes 0).	int a = 12; int c = ~a;	a = 12 → 0000 1100 ~a → 1111 0011 (assuming 8-bit representation)	c = -13 (2's complement representation)
Left Shift	<<	Shifts bits of the left operand to the left by the specified number of positions. Right side is filled with 0s.	int a = 12; int c = a << 1;	a = 12 → 0000 1100 a << 1 → 0001 1000	c = 24
Right Shift	>>	Shifts bits of the left operand to the right by the specified number of positions. Left side is filled with 0s.	int a = 12; int c = a >> 1;	a = 12 → 0000 1100 a >> 1 → 0000 0110	c = 6

Note: In C, bitwise operators work on integer types (int, unsigned int, short, char, etc.). The binary examples above assume 4-bit representation for simplicity.

Operator	Name
&	Bitwise AND
	Bitwise OR
^	Bitwise XOR
~	Bitwise NOT
<<	Left Shift
>>	Right Shift

a=5, b=3

0101 a & b
1010

0001

0101 a | b
1010

0111

0101 a ^ b
1010

0111

0101 ~a
1111

1011

1100 a << 1

1000

1100 a >> 1

0110

Comma Operator

- The comma operator can be used to link the related expressions together
- Evaluation of comma operator is from left to right.

For example:

- $x = (a = 2, b = 4, a+b)$

sizeof operator

The sizeof operator is used to determine the **size of a data type or variable in bytes.**

```
#include <stdio.h>
int main(){
    printf("%d\n",sizeof(int));
    printf("%d\n",sizeof(short int));
    printf("%d\n",sizeof(long int));
    printf("%d\n",sizeof(6));
    printf("%d\n",sizeof(float));
    printf("%d\n",sizeof(double));
    printf("%d\n",sizeof(char));

    return 0;
}
```

- Returns the size in **bytes.**
- The result type is `size_t`.
- Size may vary depending on the **compiler and system architecture.**
- No parentheses are required when used with a variable: `sizeof x` is valid.

~~Conditional operators~~

Ternary Operator (?:) in C

- The **ternary operator** is a shorthand for a simple **if-else** statement.
- It is called *ternary* because it operates on **three expressions**.
- It evaluates a condition and returns **one of two values**.

Syntax

```
condition ? expression1 : expression2;
```

Example

```
int a = 10, b = 20;  
int max = (a > b) ? a : b;
```



Conditional Operator

```
#include <stdio.h>  
int main()  
{  
    int year;  
    printf("Enter a year to check if it is a leap year or not \n");  
    scanf("%d", &year);  
    (year%4==0 && year%100!=0) ? printf("Leap year") :  
        ((year%400 ==0) ? printf("Leap Year") : printf("not a leap year"));  
}
```

Type Cast operator

The **type cast operator** is used to explicitly convert a value from one data type to another. It is written by placing the desired data type in parentheses before the value or expression. It is useful when a specific type is required for a calculation or assignment.

Syntax

(data_type) expression

Example

```
int a = 5, b = 2;  
float result = (float)a / b; // result = 2.5
```

Key Point:

Type casting temporarily changes the **data type of an expression**, without changing the original variable.

Precedence and Associativity

- **Precedence** determines **which operator is given priority** when an expression contains multiple operators.
- **Associativity** determines the **order of evaluation** when operators have the same precedence.
- Together, they help the compiler decide **how a complex expression should be evaluated**.
- Precedence is generally considered **first**, followed by associativity when needed.

```
//find x
x = 4 * 4 - 3 * 2 + 4 / 2
= (4 * 4) - 3 * 2 + 4 / 2    [* high priority operator]
= 16 - 3 * 2 + 4 / 2
= 16 - (3 * 2) + 4 / 2      [* high priority operator]
= 16 - 6 + 4 / 2
= 16 - 6 + (4 / 2)          [/ high priority operator]
= 16 - 6 + 2
= (16 - 6) + 2              [- low priority operator]
= 10 + 2
= (10 + 2)                  [+ low priority operator]
= 12
```

↓
PUMA'S REBL
TAC
↓
TAU

Operator	Description	Precedence level	Associativity
() [] → .	Function call Array subscript Arrow operator Dot operator	1	Left to Right
+ - ++ -- ! ~ * & (datatype) sizeof	Unary plus Unary minus Increment Decrement Logical NOT One's complement Indirection Address Type cast Size in bytes	2	Right to Left
* / %	Multiplication Division Modulus	3	Left to Right
+ -	Addition Subtraction	4	Left to Right
<< >>	Left shift Right shift	5	Left to Right
< <= > >=	Less than Less than or equal to Greater than Greater than or equal to	6	Left to Right
= !=	Equal to Not equal to	7	Left to Right
& ^	Bitwise AND Bitwise XOR	8 9	Left to Right
 && 	Bitwise OR Logical AND Logical OR	10 11 12	Left to Right
?:	Conditional operator	13	Right to Left
= *= /= %= += -= &= ^= = <<= >>=	Assignment operators	14	Right to Left
,	Comma operator	15	Left to Right

Expressions

An **expression** is a combination of variables, constants, operators, and function calls that evaluates to a **value**.

Common Types of Expressions:

- **Arithmetic:** $a + b * c$
- **Relational:** $a > b$
- **Logical:** $a > 0 \ \&\& \ b > 0$
- **Assignment:** $x = 10$
- **Conditional:** $\text{max} = (a > b) ? a : b$

Key Point: Every expression produces a value, which can be assigned, compared, or used as part of another expression.

Data types

Defines the type of data a variable can hold and the exact number of bytes allocated in memory

Data Type	Memory Space	Range Of Values	Format Specifiers
Signed char	1 Byte	-128 To +127	%c
Unsigned char	1 Byte	0 To 255	%c
signed int	2 Bytes	-32768 to +32767	%d
unsigned int	2 Bytes	0 to 65535	%u
float	4 Bytes	-3.4 E38 To +3.4E38	%f
double	8 Bytes	-1.7E308 To +1.7E308	%lf

Data types

Primary Data Types: These data types are predefined into C system. Their storage space, range of values, and format specifiers are all fixed.

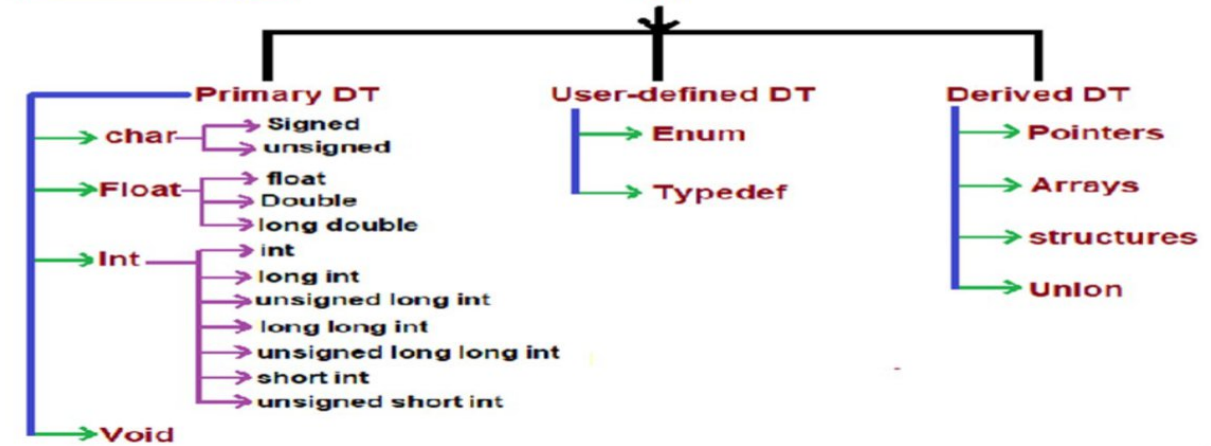
Derived Data Types : The data types which are derived from primary data types are called derived data types.

User Defined Data Types : These data types are defined by users according to their requirements with the combination of primary data types.

Data types

DT - Data type

Data Types in C



Input output functions in C

Input output functions in C are parts of `stdio.h` header file. While the initial two are used for printing or scanning a value of any data format but the later two are particularly for taking a single character or displaying a single character on console.

- `printf("%d, %c, %f", a, b, c);`
- `scanf("%d: %c: %f", &a, &b, &c);`
- `char ch = getchar();`
- `putchar(ch);`

INPUT AND OUTPUT FUNCTIONS IN C

In order to read data from standard input device keyboard and display data on standard output device monitor, 'C' offers us a collection of inbuilt functions.

These I/O functions can be categorized into two types:

- Unformatted I/O
- Formatted I/O

Unformatted input and output functions = only work with character data type. Format specifiers are not allowed to use.

`ex = getchar()` - This function is used to read a single character from keyboard .

`putchar()` - This function is used to print a single character on standard O/P screen.

In **formatted I/O functions** format specifiers are used to format the I/O before reading or printing it.

Functions	Description
<code>scanf()</code>	This is formatted input function used to read one or many Inputs from standard input device keyboard
<code>printf()</code>	This is formatted output function used to print one or many Values on standard output device monitor .

Format Specifiers

Format specifiers are special symbols used with input/output functions in C to specify the **type of data** being read or displayed.

They tell the compiler how a value should be **interpreted, formatted, or printed**.

The appropriate format specifier depends on the **data type** of the value.

- **%c** – Character (char)
- **%d** – Signed integer (int)
- **%u** – Unsigned integer (unsigned int)
- **%f** – Floating-point number (float)
- **%lf** – Double-precision floating-point number (double)
- **%s** – String (character array)
- **%x** – Hexadecimal integer
- **%o** – Octal integer

Indentation

Indentation is the practice of adding spaces or tabs at the beginning of lines to clearly show the structure and nesting of a program. It improves readability but **does not affect program execution** in C.

Why is Indentation Important?

- Makes code **easy to read and understand**.
- Clearly shows the scope of **loops, functions, and conditional statements**.
- Helps identify matching { } (curly braces).
- Reduces the chances of logical and maintenance errors.
- Encourages consistent coding practices among developers.

Best Practices

- Use **4 spaces** (or one tab) for each level of indentation.
- Indent all statements inside { } blocks.
- Keep indentation consistent throughout the program.
- Avoid mixing tabs and spaces in the same project.

Comments

- **Comments** are non-executable statements used to explain or document the code.
- They are **ignored by the compiler** and do not affect program execution.
- Help improve **code readability, understanding, and maintenance**.

Single line comments

```
// This is a single-line comment  
int a = 10;
```

Multiple line comments

```
/* This is a  
   multi-line comment */  
int a = 10;
```

Headers

What are header files

- **Header files** contain predefined **functions, macros, constants, and declarations** that can be used in a C program.
- They are included using the **#include** preprocessor directive.
- Header files help us **reuse existing functionality** instead of writing everything from scratch.

Common header files

- **stdio.h** → printf(), scanf(), getchar(), putchar()
- **stdlib.h** → malloc(), free(), rand(), exit()
- **string.h** → strlen(), strcpy(), strcmp()
- **math.h** → sqrt(), pow(), ceil(), floor()

```
#include <stdio.h>

int main() {
    printf("Hello World");
    return 0;
}
```

User Defined Header Files : User can write their own header files and can include into C programs . The user defined header files are included in “ ” (Double quotes)

predefined header files : are those which are inbuilt into C system like stdio.h. Inbuilt header files are included in < > pair as :
#include<stdio.h>

Conditional Statements

Conditional statements are control statements that allow a program to **make decisions and execute different blocks of code based on whether a given condition is true or false.**

They help a program **decide what to do** based on a condition.

Syntax

```
if(condition){  
    statement 1;  
    statement 2;  
    .  
    .  
    .  
    statement n;  
}  
else{  
    statement 1;  
    statement 2;  
    .  
    .  
    .  
    statement n;  
}
```

if Statement

Definition: Runs a block of code **only if** a given condition is true. If the condition is false, the program simply skips it.

```
if (age >= 18) {  
    printf("You can vote.");  
}
```

if-else Statement

Definition: Gives two choices. It runs one block of code if the condition is **true**, and a different block if the condition is **false**.

```
if (marks >= 40) {  
    printf("Pass");  
} else {  
    printf("Fail");  
}
```

if-else if-else Ladder

Definition: Used to check **multiple conditions in order**. The program checks conditions top to bottom and runs the block for the first condition that turns out true.

```
if (marks >= 90) {  
    printf("Grade A");  
} else if (marks >= 75) {  
    printf("Grade B");  
} else {  
    printf("Grade C");  
}
```

Switch

The **switch statement** is a selection/control statement used to choose **one block of code from multiple alternatives**.

- The value of an expression is compared with different **case values**.
- When a matching case is found, its statements are executed.
- **break** terminates the switch and prevents execution of the next cases.
- **default** executes when no case matches.

Code example

```
int day = 2;

switch (day)
{
    case 1: printf("Monday"); break;
    case 2: printf("Tuesday"); break;
    case 3: printf("Wednesday"); break;
    default: printf("Invalid day");
}
```

Switch Statement/Switch Case statement

- Switch statement acts as a substitute for a long if-else-if ladder that is used to test a list of cases.
- A switch statement contains one or more case labels which are tested against the switch expression.
- When the expression match to a case then the associated statements with that case would be executed.

Important Rules of using Switch Case statement

- Switch...case only works with integral, character or enumeration constant.
- Cases can be put in any order. However, it is recommended to put them in ascending order. It increases program readability.
- Any number of statements can be used for a specific case.
- The break statement transfers the control out of switch statement.
- The break statement is optional. It transfers program flow outside of switch...case.
- Case labels must be constants.
- Case labels must be unique.

Switch vs. if else if

- The Switch statement is an extension of the if-else if statement. The switch makes one selection when there are several choices to be made.
- IF statement uses boolean expression to decide which alternative to be executed whereas SWITCH statement uses int/char expression to decide which alternative to be executed.
- IF statement increases the complexity of a program as alternatives increases.
- Speed: A switch statement might prove to be faster than ifs provided number of cases are good.